

Competitive Programming

Section 64 Editorial: Multiplicative Number Theory

Competitive Programming Course

2026-08-20

Table of contents

1	Editorial overview	3
2	A. GCD Pair Energy	4
2.1	Restatement and constraints	4
2.2	Finding the transform	4
2.3	Worked trace	4
2.4	Algorithm	5
2.5	Correctness proof	5
2.6	Complexity and tests	5
2.7	Full C++ solution	5
2.8	Full Python solution	6
3	B. Summatory Totient	8
3.1	Why the ordinary sieve is insufficient	8
3.2	Derivation	8
3.3	Algorithm	8
3.4	Correctness proof	8
3.5	Complexity and tests	9
3.6	Full C++ solution	9
3.7	Full Python solution	10
4	C. LCM Pair Spectrum	12
4.1	Observation	12
4.2	Trace	12
4.3	Algorithm	12
4.4	Correctness proof	12
4.5	Complexity and tests	13
4.6	Full C++ solution	13
4.7	Full Python solution	13
5	D. Square-Free Rank	15
5.1	From a slow recognizer to a prefix counter	15
5.2	Trace	15
5.3	Algorithm	15
5.4	Correctness proof	16
5.5	Complexity and tests	16
5.6	Full C++ solution	16
5.7	Full Python solution	17
6	Stress-testing strategy	19
7	What to carry forward	20

1 Editorial overview

These problems share one discipline: define the cumulative family before inverting it. A formula containing μ , ϕ , or a divisor loop is not yet a solution; the proof must say which objects each term counts.

Problem	Cumulative family	Recovery step
A	pairs whose two values are divisible by d	expand $\gcd^k$ over its divisors
B	totients weighted by the number of multiples	isolate the summatory function and group quotients
C	pairs whose lcm divides m	subtract exact proper-divisor counts
D	integers divisible by each square d^2	Mobius inclusion-exclusion, then binary search

2 A. GCD Pair Energy

2.1 Restatement and constraints

We need the sum of $\gcd(a_i, a_j)^k$ over all $i < j$. Enumerating pairs is quadratic; factorizing each pair does not repair that. The value bound $M \leq 200000$ suggests sieve loops over all multiples.

2.2 Finding the transform

Define $J[d]$ by the divisor identity

$$x^k = \sum(J[d] \text{ for } d \text{ dividing } x).$$

We can compute it without memorizing a closed form. Start with $J[x] = x^k$ and subtract $J[d]$ from every proper multiple of d , processing d increasingly. When $J[x]$ is reached, every proper-divisor contribution has been removed.

For one pair with $\gcd g$, its energy becomes

$$g^k = \sum(J[d] \text{ for } d \text{ dividing } g).$$

The condition $d \mid \gcd(a_i, a_j)$ is exactly “ d divides both values.” If $\text{cnt}[d]$ input values are multiples of d , precisely $C(\text{cnt}[d], 2)$ pairs receive the contribution $J[d]$.

2.3 Worked trace

For 2,4,6 and $k=2$, inversion of x^2 begins

$$\begin{aligned} J[1] &= 1 \\ J[2] &= 4 - J[1] = 3 \\ J[3] &= 9 - J[1] = 8 \\ J[4] &= 16 - J[1] - J[2] = 12 \end{aligned}$$

All three values are divisible by 2, so $J[1] * C(3, 2) + J[2] * C(3, 2)$ already gives $3+9=12$. Larger divisors contain fewer than two values.

2.4 Algorithm

1. Build value frequencies.
2. For every d , sum frequencies at $d, 2d, 3d, \dots$ into $\text{cnt}[d]$.
3. Set $J[d] = d^k$; invert its divisor sums in increasing order.
4. Add $J[d] * \text{cnt}[d] * (\text{cnt}[d] - 1) / 2$ for every d .

2.5 Correctness proof

Lemma 1. After the inversion loop, x^k equals the sum of $J[d]$ over all divisors d of x .

Proof. When processing x , its initial value is x^k . Each already finalized proper divisor d subtracts $J[d]$ exactly once. Therefore the remaining value is the unique amount required to make the divisor sum equal x^k . Induction over increasing x proves the identity. QED.

Lemma 2. $C(\text{cnt}[d], 2)$ is the number of pairs whose gcd is divisible by d .

Proof. Their gcd is divisible by d exactly when both values are divisible by d ; choosing two of the $\text{cnt}[d]$ qualifying indices counts each pair once. QED.

Theorem. The algorithm prints the required energy sum.

Proof. By Lemma 1, each pair of gcd g contributes $J[d]$ once for every $d|g$. By Lemma 2, the outer sum counts exactly all pairs receiving each such contribution. Thus every pair contributes exactly g^k . QED.

2.6 Complexity and tests

Both sieve loops cost $O(M \log M)$ and memory is $O(M)$. Modular powers add $O(M \log k)$. Test $k=0$, all ones, all equal values, and values with several shared divisors.

2.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
constexpr int64 MOD = 1'000'000'007;

int64 mod_pow(int64 base, long long exponent) {
    int64 result = 1;
    while (exponent > 0) {
        if (exponent & 1) result = result * base % MOD;
        base = base * base % MOD;
        exponent >>= 1;
    }
    return result;
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long k;
    if (!(cin >> n >> k)) return 0;
    vector<int> frequency(1, 0);
    int maximum = 0;
    vector<int> values(n);
    for (int &value : values) {
        cin >> value;
        maximum = max(maximum, value);
    }
    frequency.resize(maximum + 1);
    for (int value : values) ++frequency[value];

    vector<int> divisible(maximum + 1);
    for (int d = 1; d <= maximum; ++d)
        for (int multiple = d; multiple <= maximum; multiple += d)
            divisible[d] += frequency[multiple];

    // jordan[d] is defined by  $x^k = \sum_{d|x} jordan[d]$ .
    vector<int64> jordan(maximum + 1);
    for (int d = 1; d <= maximum; ++d) jordan[d] = mod_pow(d, k);
    for (int d = 1; d <= maximum; ++d)
        for (int multiple = 2 * d; multiple <= maximum; multiple += d) {
            jordan[multiple] -= jordan[d];
            if (jordan[multiple] < 0) jordan[multiple] += MOD;
        }

    int64 answer = 0;
    for (int d = 1; d <= maximum; ++d) {
        int64 pairs = 1LL * divisible[d] * (divisible[d] - 1) / 2 % MOD;
        answer = (answer + jordan[d] * pairs) % MOD;
    }
    cout << answer << '\n';
}

```

2.8 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))

```

```

if not data:
    return
n, k = data[:2]
values = data[2 : 2 + n]
maximum = max(values)
frequency = [0] * (maximum + 1)
for value in values:
    frequency[value] += 1

divisible = [0] * (maximum + 1)
for d in range(1, maximum + 1):
    divisible[d] = sum(frequency[d::d])

jordan = [0] + [pow(d, k, MOD) for d in range(1, maximum + 1)]
for d in range(1, maximum + 1):
    contribution = jordan[d]
    for multiple in range(2 * d, maximum + 1, d):
        jordan[multiple] = (jordan[multiple] - contribution) % MOD

answer = 0
for d in range(1, maximum + 1):
    count = divisible[d]
    pairs = count * (count - 1) // 2
    answer = (answer + jordan[d] * pairs) % MOD
print(answer)

if __name__ == "__main__":
    main()

```

3 B. Summatory Totient

3.1 Why the ordinary sieve is insufficient

A sieve to n is excellent when n is a few million. It cannot allocate an array of length 10^9 . We need a recurrence whose distinct arguments are the few values of $\text{floor}(n/i)$.

3.2 Derivation

Start from $\sum_{d|m} \phi(d) = m$ and sum over $m=1..n$:

$$\sum_{d=1..n} \phi(d) * \text{floor}(n/d) = n(n+1)/2.$$

There is another way to expose the desired prefix. Count the same left side by the quotient $q=\text{floor}(n/d)$. Equivalently, isolate the copy of every $\phi(d)$ whose multiplier includes at least one and recursively remove the extra copies. This yields

$$\Phi(n) = n(n+1)/2 - \sum_{l=2..n} \Phi(\text{floor}(n/l)).$$

Equal quotients are repeated. If $q=\text{floor}(n/l)$, it remains equal through $r=\text{floor}(n/q)$, so the whole block subtracts $(r-l+1)*\Phi(q)$.

For $n=10$, the blocks starting at two are

$$[2,2] \rightarrow 5, [3,3] \rightarrow 3, [4,5] \rightarrow 2, [6,10] \rightarrow 1.$$

3.3 Algorithm

Precompute Φ through 10^6 . For a larger argument, initialize its answer to the triangular number, traverse quotient blocks starting at 2, recursively subtract each block contribution, and memoize the result.

3.4 Correctness proof

Lemma 1. The ungrouped recurrence for $\Phi(n)$ is valid.

Proof. In the divisor identity summed through n , $\phi(d)$ occurs once for each multiple of d , hence $\text{floor}(n/d)$ times. Removing the first occurrence of every $\phi(d)$ leaves the extra occurrences; regrouping them by the chosen multiple produces $\sum_{l=2..n} \Phi(\text{floor}(n/l))$. The remaining first copies sum to $\Phi(n)$. QED.

Lemma 2. The grouped loop equals the ungrouped sum.

Proof. The formula for r gives the maximal consecutive interval on which $\text{floor}(n/i)=q$. Multiplying $\text{Phi}(q)$ by its interval length accounts for every index in that interval once. The intervals are disjoint and cover $2..n$. QED.

Theorem. Every returned value is $\text{Phi}(n)$ modulo the modulus.

Proof. Small values are exact sieve prefixes. For a larger value, all recursive arguments are smaller. Strong induction with Lemmas 1 and 2 proves the computed recurrence exact; memoization changes only repeated work. QED.

3.5 Complexity and tests

The precomputation uses $O(B \log \log B)$ time and $O(B)$ memory for $B=10^6$. Harmonic blocking and memoization give the standard sublinear summatory-function behavior; for this bound it is comfortably below a linear sieve to n . Test around B , repeated queries, and compare with a direct totient sieve for random $n \leq 500$.

3.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
constexpr int LIMIT = 1'000'000;
constexpr int64 MOD = 1'000'000'007;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    vector<int64> phi(LIMIT + 1);
    iota(phi.begin(), phi.end(), 0);
    for (int p = 2; p <= LIMIT; ++p)
        if (phi[p] == p)
            for (int multiple = p; multiple <= LIMIT; multiple += p)
                phi[multiple] -= phi[multiple] / p;
    for (int i = 1; i <= LIMIT; ++i)
        phi[i] = (phi[i] + phi[i - 1]) % MOD;

    unordered_map<int64, int64> memo;
    function<int64(int64)> summatory = [&](int64 n) -> int64 {
        if (n <= LIMIT) return phi[n];
        if (auto it = memo.find(n); it != memo.end()) return it->second;
        int64 answer = n % MOD * ((n + 1) % MOD) % MOD;
        answer = answer * ((MOD + 1) / 2) % MOD;
        for (int64 left = 2, right; left <= n; left = right + 1) {
            int64 quotient = n / left;
```

```

        right = n / quotient;
        int64 width = (right - left + 1) % MOD;
        answer = (answer - width * summatory(quotient)) % MOD;
    }
    if (answer < 0) answer += MOD;
    return memo[n] = answer;
};

int q;
cin >> q;
while (q-- > 0) {
    int64 n;
    cin >> n;
    cout << summatory(n) << '\n';
}
}

```

3.7 Full Python solution

```

import sys

MOD = 1_000_000_007
LIMIT = 1_000_000

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return

    phi = list(range(LIMIT + 1))
    for prime in range(2, LIMIT + 1):
        if phi[prime] == prime:
            for multiple in range(prime, LIMIT + 1, prime):
                phi[multiple] -= phi[multiple] // prime
    for value in range(1, LIMIT + 1):
        phi[value] = (phi[value] + phi[value - 1]) % MOD

    memo: dict[int, int] = {}

    def summatory(n: int) -> int:
        if n <= LIMIT:
            return phi[n]
        if n in memo:
            return memo[n]
        answer = n * (n + 1) // 2 % MOD
        left = 2
        while left <= n:

```

```

        quotient = n // left
        right = n // quotient
        answer -= (right - left + 1) * summatory(quotient)
        answer %= MOD
        left = right + 1
    memo[n] = answer
    return answer

print("\n".join(str(summatory(n)) for n in data[1 : 1 + data[0]]))

if __name__ == "__main__":
    main()

```

4 C. LCM Pair Spectrum

4.1 Observation

For a fixed m , let $\text{inside}[m]$ be the number of input values dividing m . Any pair chosen among them has lcm dividing m , and every pair whose lcm divides m has both values dividing m . Therefore

$$C(\text{inside}[m], 2) = \sum(\text{exact}[d] \text{ for } d \text{ dividing } m).$$

This equivalence is the entire reduction. It changes a two-value operation (lcm) into two one-dimensional divisor transforms.

4.2 Trace

For 1,2,3,6, $\text{inside}[6]=4$, so six pairs have lcm dividing six. We already recover one pair at lcm two and one at lcm three; none has lcm one. Subtracting these proper-divisor spectra leaves four pairs at lcm six.

4.3 Algorithm

1. Sieve each frequency into all of its multiples to obtain inside .
2. Initialize $\text{exact}[m]=C(\text{inside}[m], 2)$.
3. In increasing order, subtract $\text{exact}[d]$ from every proper multiple of d .

4.4 Correctness proof

Lemma 1. Before inversion, $\text{exact}[m]$ (temporarily) counts pairs whose lcm divides m .

Proof. As shown above, this family is exactly the pairs drawn from values that divide m , of which there are $C(\text{inside}[m], 2)$. QED.

Lemma 2. After processing index m , $\text{exact}[m]$ counts pairs whose lcm is exactly m .

Proof. By increasing induction, every exact count for a proper divisor is already known and subtracted once. Lemma 1 says the cumulative count contains only exact lcms dividing m ; the remainder is exactly lcm m . QED.

The theorem follows from Lemma 2 for every output position.

4.5 Complexity and tests

Time is $O(M \log M + n)$ and memory is $O(M)$. Test repeated equal values, many ones, prime values, and an M larger than every input value.

4.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
constexpr int64 MOD = 1'000'000'007;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, maximum;
    if (!(cin >> n >> maximum)) return 0;
    vector<int> frequency(maximum + 1);
    for (int i = 0, value; i < n; ++i) {
        cin >> value;
        ++frequency[value];
    }

    vector<int> inside(maximum + 1);
    for (int divisor = 1; divisor <= maximum; ++divisor)
        for (int multiple = divisor; multiple <= maximum; multiple += divisor)
            inside[multiple] += frequency[divisor];

    vector<int64> exact(maximum + 1);
    for (int value = 1; value <= maximum; ++value)
        exact[value] = 1LL * inside[value] * (inside[value] - 1) / 2 % MOD;
    for (int divisor = 1; divisor <= maximum; ++divisor)
        for (int multiple = 2 * divisor; multiple <= maximum; multiple += divisor) {
            exact[multiple] -= exact[divisor];
            if (exact[multiple] < 0) exact[multiple] += MOD;
        }

    for (int value = 1; value <= maximum; ++value)
        cout << exact[value] << (value == maximum ? '\n' : ' ');
}
```

4.7 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, maximum = data[:2]
    frequency = [0] * (maximum + 1)
    for value in data[2 : 2 + n]:
        frequency[value] += 1

    inside = [0] * (maximum + 1)
    for divisor in range(1, maximum + 1):
        amount = frequency[divisor]
        if amount:
            for multiple in range(divisor, maximum + 1, divisor):
                inside[multiple] += amount

    exact = [count * (count - 1) // 2 % MOD for count in inside]
    for divisor in range(1, maximum + 1):
        contribution = exact[divisor]
        for multiple in range(2 * divisor, maximum + 1, divisor):
            exact[multiple] = (exact[multiple] - contribution) % MOD
    print(*exact[1:])

if __name__ == "__main__":
    main()

```

5 D. Square-Free Rank

5.1 From a slow recognizer to a prefix counter

Checking every integer and factorizing it would require work proportional to the answer. Instead count how many valid integers are at most X .

An integer is divisible by no prime square. Inclusion-exclusion over prime squares can be packaged by the Mobius function:

```
count(X) = sum mu(d) * floor(X/(d*d)), for d*d <= X.
```

If d is square-free with t prime factors, $\mu(d)=(-1)^t$, exactly the inclusion-exclusion sign for being divisible by those t prime squares. If d itself contains a square, $\mu(d)=0$, so a repeated condition contributes nothing.

5.2 Trace

For $X=10$:

```
d=1: +10
d=2: -floor(10/4) = -2
d=3: -floor(10/9) = -1
total: 7
```

Indeed 1,2,3,5,6,7,10 are the seven square-free values.

5.3 Algorithm

Sieve μ up to the square root of the largest binary-search upper bound. For every rank, binary-search the smallest X for which $\text{count}(X) \geq k$. $3k$ is a safe course bound; the density of square-free integers is bounded away from zero, and the implementation can also double the bound if desired.

5.4 Correctness proof

Lemma 1. $\text{count}(X)$ equals the number of square-free integers at most X .

Proof. Fix an integer x . Its total coefficient in the sum is $\sum_{d^2|x} \mu(d)$. This is one when no prime square divides x , and zero otherwise by inclusion-exclusion. Summing those indicators over $x \leq X$ gives the formula. QED.

Lemma 2. The binary search returns the k -th square-free integer.

Proof. $\text{count}(X)$ is monotone. At the returned smallest X , at least k valid values are no larger than X , while fewer than k are no larger than $X-1$. Therefore X itself is precisely the k -th. QED.

5.5 Complexity and tests

For one query, each count costs $O(\sqrt{X})$ and binary search uses $O(\log X)$ counts. The Möbius sieve uses $O(\sqrt{\max X})$ memory. Test ranks surrounding perfect squares and compare the first several hundred answers with trial division.

5.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    vector<int64> queries(q);
    int64 largest = 0;
    for (int64 &k : queries) {
        cin >> k;
        largest = max(largest, k);
    }

    int limit = sqrtl(3.0L * largest) + 2;
    vector<int> mu(limit + 1), primes;
    vector<bool> composite(limit + 1);
    mu[1] = 1;
    for (int value = 2; value <= limit; ++value) {
        if (!composite[value]) {
            primes.push_back(value);
            mu[value] = -1;
        }
    }
    for (int prime : primes) {
```

```

        if (1LL * value * prime > limit) break;
        composite[value * prime] = true;
        if (value % prime == 0) {
            mu[value * prime] = 0;
            break;
        }
        mu[value * prime] = -mu[value];
    }
}

auto count_squarefree = [&](int64 x) {
    int64 result = 0;
    for (int64 d = 1; d * d <= x; ++d)
        result += mu[d] * (x / (d * d));
    return result;
};

for (int64 k : queries) {
    int64 low = 1, high = 3 * k;
    while (low < high) {
        int64 middle = low + (high - low) / 2;
        if (count_squarefree(middle) >= k)
            high = middle;
        else
            low = middle + 1;
    }
    cout << low << '\n';
}
}

```

5.7 Full Python solution

```

import math
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    queries = data[1 : 1 + data[0]]
    limit = math.isqrt(3 * max(queries)) + 1

    mu = [0] * (limit + 1)
    composite = bytearray(limit + 1)
    primes: list[int] = []
    mu[1] = 1
    for value in range(2, limit + 1):

```

```

    if not composite[value]:
        primes.append(value)
        mu[value] = -1
    for prime in primes:
        product = value * prime
        if product > limit:
            break
        composite[product] = 1
        if value % prime == 0:
            mu[product] = 0
            break
        mu[product] = -mu[value]

def count_squarefree(x: int) -> int:
    return sum(
        mu[d] * (x // (d * d))
        for d in range(1, math.isqrt(x) + 1)
    )

answers = []
for rank in queries:
    low, high = 1, 3 * rank
    while low < high:
        middle = (low + high) // 2
        if count_squarefree(middle) >= rank:
            high = middle
        else:
            low = middle + 1
    answers.append(str(low))
print("\n".join(answers))

if __name__ == "__main__":
    main()

```

6 Stress-testing strategy

The supplied randomized tests deliberately use tiny domains and independent brute definitions: explicit pairs for A and C, direct gcd counts for phi in B, and trial square divisibility in D. This is stronger than comparing two implementations of the same optimized formula.

7 What to carry forward

When a hard arithmetic problem asks for exact gcd/lcm classes, first count an easier cumulative divisibility family. When `floor(n/i)` appears, enumerate its constant intervals. When the target is the k-th valid integer, search on a proven prefix-count oracle.